Introducción

El redimensionamiento de imágenes es un problema complejo y muy importante en el procesamiento digital de imágenes. Los métodos más sencillos presentan limitaciones evidentes: el recorte, que elimina contenido de los bordes, corre el riesgo de descartar partes centrales de la imagen, mientras que el escalado uniforme tiende a distorsionar los objetos. En este contexto, el Seam-carving es un tipo de recorte de imágenes formulado por Shai Avidan y Ariel Shamir[1] que propone una solución diferente. Este método se basa en la eliminación de costuras de mínima energía, es decir, caminos de píxeles con poca importancia para la imagen. Este redimensionamiento se adapta al contenido de la imagen, por lo que conserva mejor las características más importantes de la misma.

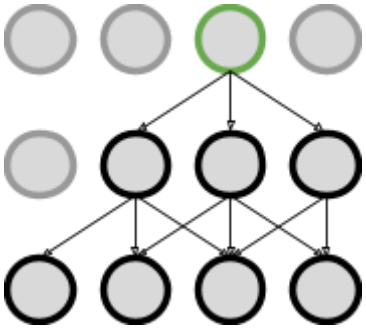
El objetivo de este trabajo es implementar (en C++) y analizar distintos algoritmos (Fuerza Bruta, Backtracking y Programación Dinámica) para resolver el problema de encontrar la costura de mínima energía a partir de una matriz que representa la energía de cada píxel de una imagen. La idea es comparar el funcionamiento y el rendimiento de los tres algoritmos de forma tanto teórica como experimental. Además, se realiza una reimplementación de estos algoritmos en Python (utilizando herramientas de IA) con el fin de comparar este lenguaje con C++.

El informe se organiza en cuatro secciones principales. En la primera, se presentan los algoritmos implementados junto con su descripción, pseudocódigo y análisis de complejidad. En la segunda se discuten aspectos relacionados con las decisiones de implementación y diseño. En la tercera se detalla el proceso de experimentación, donde se comparan los algoritmos y se analizan sus tiempos de ejecución bajo distintas condiciones. En la última se presentan las conclusiones del trabajo, incluyendo observaciones, limitaciones, recomendaciones y posibles mejoras.

Descripción del problema

Modelado

Para resolver el problema del seam-carving en una imagen n x m se modelan sus píxeles como celdas de una matriz n x m. A estos se les asigna un número de "energía" de acuerdo a su grado de diferencia con los píxeles adyacentes. A partir de esta nueva matriz de energías se busca un camino (*seam* o costura) de mínima energía entre píxeles de la primera fila hasta la última entendiendo que dónde haya menor variabilidad entre píxeles el contenido será menos relevante. Para encontrar dicha costura, se modelan los caminos posibles a partir de cada nodo de la primera fila como un grafo en el que cada vértice tiene únicamente una arista con el vértice que se encuentra directamente debajo de él, y a la izquierda y derecha de éste. Entonces, cada vértice es un píxel. Este modelo es el que utilizamos para luego pensar los algoritmos de fuerza bruta, backtracking y programación dinámica que más tarde implementaremos.



En el caso de la programación dinámica, además, es necesario definir cómo es posible resolver el problema de encontrar la costura de mínima energía a partir de otros subproblemas. De este modo, definimos:

> *Para M = matriz de energías n x m con $e_{i,j}$=energía del píxel de la fila i columna j (ambas indexadas desde 1) definimos PD(M, i, j) como la energía total de la costura de mínima energía partiendo del píxel de la fila i columna j hacia abajo.*

$$PD(M,i,j) = \begin{cases} e_{i,j} & \text{si } i = n \\ e_{i,j} + \min_{\substack{k\in\{-1,0,1\} \\ 1\leq j+k\leq m}}(f(i+1,j+k)) & \text{en caso contrario} \end{cases}$$

A partir de esta función es posible definir una función de programación dinámica en la que se guardan los resultados de la función para cada par de índices i,j.

[1] https://perso.crans.org/frenoy/matlab2012/seamcarving.pdf

Diseño e Implementación de los algoritmos

Fuerza Bruta

La solución de fuerza bruta consiste en generar todos los seams posibles y quedarse con aquel que tenga menor energía total. Para esto, se construyen recursivamente todas las combinaciones válidas respetando la restricción de adyacencia entre filas. No se podan las soluciones no factibles, es decir que no respetan esta condición, sino que simplemente se diseña la fuerza bruta de forma "inteligente" para que no trate de recorrer estos caminos. Así, la función recursiva va construyendo un vector V que representa el seam parcial, donde cada posición indica la columna elegida en cada fila.

El algoritmo construye el seam fila por fila con la siguiente secuencia de pasos: en la primera fila se prueban todas las columnas posibles. En las filas siguientes, sólo se consideran las columnas adyacentes a la elegida anteriormente (izquierda, misma columna o derecha). En cada llamado recursivo, se genera un seam parcial V que se extiende hasta la última fila de la matriz de energías. Cuando se alcanza la última fila (k == energia.size()), se obtiene un seam completo. Luego, se compara su energía con la mejor encontrada hasta el momento utilizando la función energia_calc().

Este algoritmo explora exhaustivamente todo el espacio de soluciones posibles. Realizamos una versión del pseudocódigo de nuestra implementación siguiendo las convenciones de los pseudocódigos vistos en clase:

```
SeamFB(energia, k, V)
    if k = ultima fila then
      return V
    else
      if k = 0 then
        mejor ← vacío
        for cantidad de columnas do
          V' ← V + [i]
          S ← SeamFB(energia, k+1, V')
          if mejor es vacío o energia(S) < energia(mejor) then
            mejor ← S
        return mejor
      else
        mejor ← vacío
        for las columnas adyacentes do
          if columna válida then
            V' ← V + [j]
            S ← SeamFB(energia, k+1, V')
            if mejor es vacío o energia(S) < energia(mejor) then
              mejor ← S
        return mejor
```

La mayor dificultad que surgió al diseñar el algoritmo de Fuerza Bruta fue manejar correctamente el tratamiento diferencial de la primera fila de la matriz de energía respecto de las siguientes. Al principio, se buscó unificar el comportamiento en un solo caso recursivo, sin embargo, este enfoque resultó problemático, ya que no se encontró una forma clara de generar todas las soluciones factibles sin recorrer todas las columnas en la primera iteración. Esto se debe a que la primera fila es la única en la que no hay una decisión anterior que limite las posibilidades: es necesario considerar todas las columnas como inicios válidos del *seam*. Esta diferencia obligó a separar el caso $k = 0$ del resto.

<u>Complejidad</u>

Si llamamos A a la cantidad de filas (altura de la imagen) y B a la cantidad de columnas (ancho de la imagen), sabemos que en la primera fila hay B opciones. Luego, para avanzar una fila tengo 3 opciones por cada píxel:

izquierda, misma columna y derecha (salvo para la primera y última columna, pero al no ser un cambio significativo no lo contemplamos). Por tanto, la cantidad total de seams posibles es $B \cdot 3^{A-1}$ (se excluye la primera fila del exponente). Además, para cada seam, nuestro algoritmo calcula su energía, lo cual tiene costo O(A) (se incluye una columna por fila en la respuesta). Por tanto, nuestro algoritmo de FB tiene

$$O(B \cdot 3^{A-1} \cdot A)$$

Backtracking

La solución de Backtracking se basa en la misma idea que Fuerza Bruta: generar seams válidos fila por fila. Sin embargo, a diferencia de FB, este algoritmo incorpora una poda por optimalidad que evita explorar ramas que no pueden mejorar la mejor solución encontrada hasta el momento.

Además, en BT, a diferencia de FB, se mantienen parámetros de entrada adicionales para ejecutar la poda: energia_actual, que representa la energía acumulada del seam parcial, mejor_energia, que representa la mejor energía encontrada hasta el momento, y mejor_solución, la mejor solución encontrada hasta el momento. Exceptuando estos detalles, la lógica general es muy similar al de Fuerza Bruta: en cada paso, se construye un seam parcial V y se acumula su energía en energia_actual.

Entonces, las diferencias claves con FB están en dos puntos:

1. <u>Poda por optimalidad</u>
   Si el seam parcial ya tiene energía mayor o igual que la mejor solución encontrada, no tiene sentido seguir explorando esa rama. Esta condición es lo primero que se verifica en cada llamado recursivo.
2. <u>Actualización incremental de la energía</u>
   En lugar de recalcular la energía completa del seam cada vez, se va acumulando a medida que se construye. Esto elimina la necesidad de utilizar la función energia_calc() para cada solución posible.

Cuando se alcanza la última fila, se compara la energía acumulada con la mejor encontrada y se actualiza mejor_energia si corresponde, al igual que mejor_solución. Así como con FB, realizamos una versión del pseudocódigo:

```
SeamBT(energia, k, V, energia_actual, mejor_energia, mejor_solucion)
    if energia_actual ≥ mejor_energia then
      return
    if k = ultima fila then
      if energia_actual < mejor_energia then
        mejor_energia ← energia_actual
        mejor_solucion ← V
    else
      if k = 0 then
        for cantidad de columnas do
          V' ← V + [i]
          nueva_energia ← energia_actual + energia[k][i]
          SeamBT(energia, k+1, V', nueva_energia, mejor_energia,
          mejor_solucion)
      else
        for las columnas adyacentes do
          if columna valida then
            V' ← V + [j]
            nueva_energia ← energia_actual + energia[k][j]
            SeamBT(energia, k+1, V', nueva_energia, mejor_energia,
                mejor_solucion)
```

La mayor dificultad que apareció al diseñar el algoritmo de Backtracking fue mantener de forma consistente la mejor energía y la mejor solución encontradas a lo largo de los llamados recursivos. En un primer

momento, se intentó seguir una estructura similar a la de Fuerza Bruta, donde la función devuelve directamente un vector solución. Sin embargo, este enfoque complicaba bastante el seguimiento de la mejor solución global, ya que cada llamada trabajaba de forma más aislada y no resultaba claro cómo transferir la información entre distintos llamados recursivos. Tras identificar que era mucho más fácil llamar a una función auxiliar que tenga la mejor solución como parámetro de entrada, el diseño del algoritmo se simplificó bastante. A su vez, adoptar este enfoque para la mejor solución facilitó extrapolar esta idea al manejo de las energías: en lugar de recalcularlas en cada comparación, es mucho más eficiente mantener la energía actual y la mejor energía como parámetros de entrada que se actualizan progresivamente.

Complejidad temporal

Si seguimos llamando A a la cantidad de filas de la matriz de energía de la imagen y B a la cantidad de columnas, el peor caso para nuestro algoritmo es aquél en el que la poda nunca se activa, ya que en ese caso el algoritmo recorre el mismo espacio de soluciones que Fuerza Bruta.

Por lo tanto, la complejidad en el peor caso sigue siendo:

$$O(B \cdot 3^{A-1} \cdot A)$$

Sin embargo, en la práctica, la poda reduce significativamente la cantidad de nodos explorados. Si se encuentra una buena solución temprano, muchas ramas se descartan sin explorarse completamente. Esto hace que el rendimiento real sea muy dependiente de la instancia, en particular de la distribución de la energía, y de la forma en la que se recorren los *seams* factibles.

Programación Dinámica

Optamos por una solución de Programación Dinámica *bottom-up* que se basa en reutilizar soluciones a subproblemas para construir el *seam* óptimo sin necesidad de explicitamente explorar todas las combinaciones posibles. A diferencia de Fuerza Bruta y Backtracking, este algoritmo no genera caminos *per se*, sino que calcula de manera determinística la mejor energía posible desde cada píxel hasta la última fila. Para esto, se construye una matriz auxiliar MEMO de las mismas dimensiones que la matriz de energía, donde cada posición guarda la energía mínima acumulada desde ese píxel hasta el final de la imagen. Esta manera de ver la resolución del *seam-carving* a partir de subproblemas está basada en la función planteada en la sección del informe "Modelado".

El algoritmo funciona en dos etapas bien diferenciadas: el *tableau-filling* de abajo hacia arriba de la matriz MEMO y la reconstrucción del *seam* óptimo. Primero, se inicializa la última fila de MEMO con los valores de la matriz de energía original, ya que a partir de estos píxeles no hay otro camino que tomar. Luego, para cada píxel de las filas superiores, se calcula el costo mínimo sumando su energía con el mínimo entre sus tres posibles sucesores (abajo, abajo-izquierda y abajo-derecha). De esta forma, cada celda de MEMO contiene el costo óptimo de un seam que comienza en esa posición. Luego, una vez completada la matriz, se reconstruye el *seam* óptimo comenzando desde la primera fila. Se elige la columna con menor valor en MEMO[0] y, a partir de allí, se baja fila por fila eligiendo siempre la opción de mínima energía entre las columnas adyacentes en la fila siguiente.

El pseudocódigo para nuestra implementación es el siguiente:

```
SeamPD(energia)
    crear V
    crear MEMO de tamaño filas x columnas
    for cantidad de columnas do
        MEMO[ultima fila][j] ← energia[ultima fila][j]
    for i desde anteultima fila hasta primera fila do
        for cantidad de columnas do
```

```
        izq ← ∞
        der ← ∞
        if columna derecha es válida then
           der ← MEMO[i+1][j+1]
        if columna izquierda es válida then
           izq ← MEMO[i+1][j-1]
        medio ← MEMO[i+1][j]
        MEMO[i][j] ← energia[i][j] + min(izq, medio, der)
    mejor_col ← mínimo en MEMO[0]
    V ← [mejor_col]
    for filas restantes do
       elegir la mejor columna adyacente en MEMO[i+1]
       agregarla a V
    return V
```

La mayor dificultad que surgió al diseñar este algoritmo fue que, en un inicio, se intentó pensar en una estrategia *top-down*. El problema con este enfoque era que no resultaba claro cómo estructurar el MEMO para almacenar simultáneamente las energías óptimas y la información necesaria para reconstruir la solución. La solución terminó siendo más sencilla al adoptar un enfoque *bottom-up*. Al reinterpretar cada celda del MEMO como la mejor energía acumulada posible desde ese píxel hacia abajo, el problema se vuelve más transparente. En este esquema, la inicialización del MEMO es más fácil de encarar, ya que se puede ver que la mejor energía desde los píxeles en la última fila es simplemente su propia energía. A partir de ahí, el resto de la matriz se completa de manera sistemática, lo que simplifica tanto el cálculo como la posterior reconstrucción del *seam* óptimo. misma de ese píxel.

<u>Complejidad temporal</u>

Si seguimos llamando A a la cantidad de filas y B a la cantidad de columnas, el algoritmo comienza inicializando la última fila del MEMO y este proceso recorre una sola vez la cantidad de columnas, por lo que tiene costo $O(B)$. Luego, el llenado de la matriz MEMO utiliza dos bucles anidados: uno que recorre las filas (A) y otro que recorre las columnas (B). Dentro del bucle interno sólo se realizan operaciones O(1) (comparaciones y suma), por lo que esta parte tiene costo: $O(A \cdot B)$. Por último, la reconstrucción del óptimo primero busca el mínimo en la primera fila, lo cual tiene O(A), y luego se reconstruye el camino bajando fila por fila, lo cual tiene costo O(B). Por tanto, la reconstrucción de la solución tiene costo $O(A + B)$. Sumando todas las partes, la complejidad total queda dominada por el llenado de la matriz:

$$O(A \cdot B)$$

<u>Complejidad espacial:</u>

El algoritmo utiliza una matriz auxiliar MEMO de tamaño $A \cdot B$, donde se guarda la mejor energía posible desde cada posición hasta la última fila. Por lo tanto, el espacio utilizado es:

$$O(A \cdot B)$$

El vector utilizado para almacenar el seam resultante de tamaño A no cambia el orden de la complejidad ya que el memo significativamente más grande.

Experimentación

En esta sección detallamos las experimentaciones realizadas para el algoritmo de Seam Carving. Primero comparamos la ejecución del código en lenguaje C++ y Python. Luego analizamos si existía una diferencia significativa en el tiempo de ejecución entre imágenes a color e imágenes en blanco y negro. A continuación, estudiamos cómo evoluciona el tiempo de ejecución al partir de una imagen con fondo homogéneo y agregar progresivamente información. Finalmente, testeamos si la posición de objetos en el extremo izquierdo de las imágenes enlentece los tiempos de ejecución del algoritmo en su versión Backtracking.

<u>Experimento 1:</u> Tiempos de ejecución Python vs C++

4.1.1 Metodología

En esta sección se presenta la metodología utilizada para implementar y comparar el rendimiento de los algoritmos en C++ y Python.

Para ello, se realizó la conversión de los archivos *.cpp* de C++ a Python utilizando herramientas de asistencia como ChatGPT. Durante este proceso, se buscó mantener la lógica original del programa, adaptando únicamente aquellos aspectos necesarios por diferencias entre los lenguajes. En particular, al adaptar *Imagen.cpp* a *Imagen.py*, fue necesario reemplazar las bibliotecas *stb_image* y *stb_image_write* por la librería *Pillow* (PIL), conservando la funcionalidad de carga y guardado de imágenes. Además, surgieron algunos inconvenientes durante la ejecución que debieron corregirse de forma iterativa, como conflictos con funciones reservadas del lenguaje (por ejemplo, *min* en Python) y detalles de indexación.

Para las pruebas, se utilizó la misma imagen llevada a distintas dimensiones (10x10, 15x15 y 100x100). Se eligieron dimensiones lo suficientemente pequeñas en los casos de fuerza bruta y backtracking para permitir su ejecución en tiempos razonables." Es importante aclarar que no se realizó una comparación directa entre algoritmos, ya que se utilizó la imagen en distinto tamaño para cada caso. Por lo tanto, el análisis se centra únicamente en la diferencia de rendimiento entre lenguajes para cada algoritmo en particular.

Para medir los tiempos de ejecución, se utilizó en Python la función *time.perf_counter()*, que permite obtener mediciones precisas del tiempo transcurrido. En particular, se midió tanto el tiempo de ejecución de cada iteración del algoritmo como el tiempo total de todas las iteraciones dentro del modo imagen para poder chequear la sanidad del dato obtenido. En el caso de C++, se utilizó la biblioteca *<chrono>*, tomando marcas de tiempo antes y después del bloque de iteraciones mediante *std::chrono::high_resolution_clock*. A partir de estas mediciones se calculó el tiempo total de ejecución y el promedio por iteración . En ambos lenguajes, el tiempo total se midió abarcando todas las iteraciones solicitadas, lo que permite una comparación más representativa del rendimiento general de cada implementación.

Muestreo: Se realizaron **10 ejecuciones independientes** por cada algoritmo y lenguaje, calculando el promedio de los tiempos obtenidos.

Entorno: Los experimentos se realizaron en una notebook Dell Inspiron 16 5620, equipada con un procesador Intel Core i5-1235U (12th Gen), 16 GB de RAM y almacenamiento SSD NVMe de 512 GB (KIOXIA). El sistema operativo utilizado fue Windows 11. Durante las ejecuciones, la computadora se mantuvo conectada a la corriente y con una carga de trabajo estable (Visual Studio Code y una ventana de Chrome con una hoja de cálculo abierta). Para la implementación en C++ se utilizó el compilador GCC 15.2.0.

Hipótesis Inicial Python vs C++

Para una misma entrada y utilizando el mismo algoritmo, se espera que la implementación en C++ tenga un menor tiempo de ejecución que la implementación en Python. Además, se plantea que la diferencia relativa entre ambos lenguajes puede variar según el tipo de algoritmo y el tamaño de la entrada, siendo más notoria en casos donde se realizan una gran cantidad de operaciones o iteraciones. Esta diferencia se explica por las características propias de cada lenguaje: C++ permite una ejecución más cercana al hardware y un control más directo sobre la memoria, mientras que Python introduce mayores costos en la ejecución de operaciones debido a su forma de manejar tipos y estructuras de datos.

Resultados de los experimentos:

Algoritmo	Tamaño imagen	Iteraciones	Python Prom.	C++ Prom.	Rel. Python/C++
Fuerza Bruta	10x10	5	0.5771	0.04286749	13.46
Backtracking	15x15	7	0.6771566	0.08890415	7.62
Programación Dinámica	100x100	50	2.2134938	0.05982	36.99

Promedios Calculados con N = 10 (tiempos en segundos)

Análisis de los resultados:
En Fuerza Bruta, Python tarda **1246% más** que C++.
En Backtracking, Python tarda **662% más** que C++.
En Programación dinámica Python tarda **3599% más** que C++; máxima diferencia por eficiencia del algoritmo
En todos los casos, los resultados muestran una diferencia consistente a favor de C++, aunque la magnitud de esta diferencia varía según el algoritmo y el tamaño de la instancia.

Conclusión del experimento

A partir de los resultados obtenidos, se confirma la hipótesis inicial: para una misma entrada y utilizando el mismo algoritmo, la implementación en C++ presenta menores tiempos de ejecución que la implementación en Python.
En los tres casos analizados (fuerza bruta, backtracking y programación dinámica), Python resultó consistentemente más lento que C++, aunque en distinta magnitud. Esta diferencia se explica por las características propias de cada lenguaje, ya que C++ ofrece una ejecución más eficiente y un mejor manejo de memoria.
Además, se observa que la diferencia relativa entre ambos lenguajes no es constante, sino que varía según el caso analizado. En particular, en programación dinámica se registra la mayor diferencia, lo que sugiere que este tipo de implementación aprovecha mejor las ventajas de C++.
Los resultados muestran que, utilizando el mismo algoritmo y entrada, el lenguaje de programación tiene un impacto significativo en el tiempo de ejecución, siendo C++ más eficiente que Python en todos los casos considerados.

Experimento 2: Imágenes a color vs. imágenes en blanco y negro

Hipótesis Inicial

Nuestra hipótesis planteaba que las imágenes en blanco y negro presentarían mayores tiempos de ejecución. La intuición era que, al reducir la variedad de tonos, los valores de energía de los píxeles serían más similares entre sí, generando más "empates" o caminos con energías casi idénticas y dificultando la identificación del mínimo global.

Metodología de trabajo

Para garantizar la integridad de los datos y minimizar el impacto de variables externas (como procesos en segundo plano), se definió el siguiente protocolo:

- Muestreo**:** Se realizaron 10 ejecuciones independientes por cada configuración (tipo de imagen y cantidad de iteraciones), calculando el promedio de los tiempos. Cada ejecución se realizó bajo

condiciones similares (misma carga del sistema y computadora enchufada) para reducir el ruido. Se utilizó la biblioteca *<chrono>*, tomando marcas de tiempo antes y después del bloque de iteraciones mediante *std::chrono::high_resolution_clock*.

- Entorno de ejecución**:** Se utilizó una Notebook Lenovo Ideapad Slim 3i 15IRH8, 16 gb RAM, SSD 512GB con 13th Gen Intel(R) Core(TM) i7-13620H. La computadora estuvo permanentemente en carga de batería. Además de Visual Studio Code había una ventana abierta de Chrome con una hoja de cálculo para guardar los resultados. El sistema operativo es Windows 11. El software es C++ con GCC 15.2.0.
- Algoritmos**:** Se utilizó Programación Dinámica (PD) para instancias de alta resolución y, posteriormente, Backtracking (BT) para instancias de baja resolución.
- Instancias**:**
    - **PD:** "mariposa" (360x360), "messi" (720x744) y "taylor" (1920x1080).
    - **BT:** versiones redimensionadas a 20x20 píxeles para manejar la complejidad exponencial del algoritmo.

Resultados de Programación Dinámica (PD)

	Tipo Imagen	1 Iteración	5 Iteraciones	10 Iteraciones	20 Iteraciones	Variación (porcentaje)
Mariposa	Color	0,09431014	0,4686331	1,748725	2,698782	
Mariposa	Blanco y Negro	0,09591207	0,4682915	1,707093	2,671735	-0,4393037492
Taylor	Color	2,114596	10,6313	21,71494	41,94049	
Taylor	Blanco y Negro	2,126409	10,69362	21,31868	41,85366	-0,2217558616
Messi	Color	0,3859867	1,974845	3,863557	7,671889	
Messi	Blanco y Negro	0,384216	1,942434	3,900945	7,656084	-0,3345602561

Los resultados muestran una variación promedio global cercana a -0,40%, lo que indica que la diferencia de tiempo es prácticamente despreciable.

Al analizar la implementación (función *encontrarSeamPD_aux*), se observa que el algoritmo recorre la matriz de energía mediante bucles anidados en un esquema determinístico de *tableau-filling*. Esto implica que cada celda debe procesarse independientemente de su valor, por lo que el contenido de la imagen no influye en el tiempo de ejecución.

Análisis de Backtracking (BT)

Al darnos cuenta de que PD no depende del contenido de la imagen, nos interesó analizar si esto también ocurría en Backtracking. Consideramos que, gracias a la poda, este algoritmo sí podría verse afectado por la estructura de la matriz de energía, por lo que nuestra hipótesis podría cumplirse.

Para ello, utilizamos imágenes de 20x20 píxeles, ya que la complejidad del algoritmo hacía inviable trabajar con tamaños mayores. También limitamos las iteraciones a un máximo de 10.

	Tipo Imagen	1 Iteración	5 Iteraciones	10 Iteraciones	Variación (porcentaje)
Mariposa	Color	0,01981126364	0,3414713	13,06197	
Mariposa	Blanco y Negro	0,01397956	0,2270067	12,76531	-21,74282611
Limón	Color	0,4896087	14,80472	27,22801	

Limón	Blanco y Negro	0,2369545	6,173095	12,10246	-55,15264036
Ying Yang	Color	0,09213834	4,008541	5,383408	
Ying Yang	Blanco y Negro	0,10256551	1,832031	3,150421	-28,15300293

Los resultados contradicen nuestra hipótesis inicial: en todos los casos, las imágenes en blanco y negro resultaron más rápidas de manera significativa.

Creemos que la explicación más plausible (aunque es sólo una conjetura) se encuentra en la poda y en el cálculo de la energía de los píxeles.

Por un lado, la poda utilizada compara la energia actual con la mejor encontrada hasta el momento, terminando con la rama recursiva en el caso de que la primera sea mayor que la segunda. Esto implica que el rendimiento depende de cuán rápido se encuentre una solución de baja energía. Si esto ocurre temprano, se pueden descartar grandes porciones del espacio de búsqueda. Por otro lado, la energía de cada píxel se calcula en función de las diferencias entre los valores RGB de píxeles vecinos: el ciclo for entre las líneas 83 y 87 de Imagen.cpp recorre los tres canales RGB y para cada uno suma la diferencia entre el valor del canal del píxel para el cual se está calculando la energía y el valor de los canales de los píxeles adyacentes.

En imágenes en blanco y negro, los tres canales tienden a coincidir, por lo que se reduce la variabilidad entre píxeles. Esto podría generar una matriz de energía más homogénea y facilitar la aparición temprana de seams de baja energía, reduciendo rápidamente el valor de *mejor_energia* y activando la poda de manera más agresiva.

En contraste, en imágenes a color, la variabilidad entre canales introduce mayor irregularidad en la matriz de energía, lo que puede hacer que más caminos parciales parezcan prometedores durante más tiempo, retrasando la poda.

El caso del Ying Yang y la conversión a blanco y negro

El caso de la imagen de Ying Yang merece un análisis aparte. A diferencia de las otras imágenes, esta ya era originalmente en blanco y negro. Aun así, la versión "convertida" volvió a mostrar mejores tiempos que la original.

Este resultado es contraintuitivo y nos llevó a cuestionar el método de conversión utilizado. El sitio que utilizamos no documenta su implementación, pero investigando descubrimos que existen dos métodos principales para convertir imágenes a blanco y negro:

- <u>Promedio</u>**:** asigna a cada píxel el promedio de los canales RGB.
- <u>Luminancia</u>**:** utiliza una combinación ponderada (mayor peso al verde, menor al azul).

Para investigar esto de manera informal, generamos imágenes completamente rojas, verdes y azules. Al convertirlas, observamos que las intensidades resultantes no eran iguales (verde > rojo > azul), lo cual es coherente con un método de luminancia. En este tipo de método, la información cromática se suele resumir con una ecuación como $I = 0.299R + 0.587G + 0.114B$.

Si este fuera el caso, la conversión no sólo reduce la información cromática (al pasar de tres canales a uno), sino que también introduce una redistribución de intensidades. Al utilizar una combinación ponderada de los canales RGB, distintos colores que antes podían tener comportamientos similares pasan a diferenciarse en la escala de grises, mientras que otros contrastes pierden fuerza. Esto implica que la matriz de energía resultante no es simplemente una versión "más plana" de la original, sino que puede presentar una estructura distinta. Por tanto, este método de conversión en algunos casos podría generar regiones más homogéneas y favorecer la aparición temprana de seams de baja energía (facilitando la poda en Backtracking, como se ve en nuestra experimentación), pero en otros podría introducir nuevos contrastes o acentuar diferencias que antes

no eran tan relevantes, haciendo que la matriz de energía resulte incluso más compleja. Por lo tanto, el efecto de la conversión no es unívoco y depende tanto del método que se use para la misma como de la composición original de la imagen.

Puede que sea este funcionamiento el que esté detrás del caso del Ying Yang. Aunque la imagen ya está en escala de grises, en la práctica sigue estando almacenada en formato RGB (con tres canales). Aplicar una conversión por luminancia sobre estos valores puede no ser completamente neutro: pequeñas diferencias entre canales pueden ser re-ponderadas, generando leves cambios en las intensidades finales. Esto puede modificar sutilmente los contrastes de la imagen y, por lo tanto, la estructura de la matriz de energía. En este sentido, la conversión podría estar "simplificando" ciertas regiones (facilitando la poda) o alterando la distribución de energías de manera favorable para el algoritmo.

Tampoco se puede descartar la posibilidad de que el método de conversión incluya modificaciones independientes del color como suavizado, normalización o reducción de ruido, y que sea esta la explicación detrás del caso Ying Yang. Estas pequeñas variaciones (no relacionadas con el color de la imagen) pueden alterar la estructura de la matriz de energía y afectar el comportamiento del algoritmo.

De todos modos, estas explicaciones son conjeturas basadas en una muestra limitada y en un proceso de conversión cuyo funcionamiento sólo podemos inferir en base a testings rudimentarios.

Conclusiones y conjeturas

A partir de los resultados obtenidos, podemos concluir que:

- Independencia de información en PD: Se confirma que el algoritmo de Programación Dinámica es independiente del contenido de la imagen. Su tiempo de ejecución depende únicamente del tamaño de la matriz de entrada.
- Refutación parcial de la hipótesis: pareciera que nuestra hipótesis inicial no se cumple para Backtracking. Lejos de dificultar la búsqueda, las imágenes en blanco y negro parecen facilitar la poda, reduciendo el tiempo de ejecución.
- Sensibilidad al contenido en BT: A diferencia de PD, el algoritmo de Backtracking sí es sensible a la estructura de la matriz de energía. Pequeñas variaciones en esta estructura pueden tener un impacto significativo en el rendimiento.
- Posible influencia del método de conversión: Los resultados son compatibles con la idea de que el método de conversión a blanco y negro (posiblemente basado en luminancia) afecta la distribución de energías y, por ende, el comportamiento del algoritmo. No obstante, esta conclusión es tentativa debido a la falta de información sobre el proceso de conversión.
- Limitaciones del experimento: La dependencia de una herramienta externa no documentada y el tamaño reducido de las instancias en Backtracking limitan la generalización de los resultados. En particular, el caso del Ying Yang muestra que pequeñas transformaciones pueden producir efectos no triviales, cuya explicación requiere un análisis más profundo. Para una próxima repetición del experimento, sería prudente implementar un método de conversión de las imágenes que sea más controlable.

Experimento 3: Cantidad de objetos, posicionamiento de objetos a izquierda y tiempos de ejecución

Hipótesis Inicial

Al ser el *seam-carving* un algoritmo que recorta imágenes de manera "sensible al contenido", una de las preguntas evidentes se corresponde a qué características debe presentar éste para que el método se ejecute en un tiempo razonable. Por lo tanto, a partir de una intuición simple, que hasta un punto a más objetos el algoritmo puede reconocer más fácilmente qué contenido no cortar, propusimos la siguiente hipótesis: existe una relación inicialmente negativa que alcanza un pico y luego se vuelve positiva entre la cantidad de objetos presentes en una imagen y el tiempo que tarda la ejecución de un algoritmo de *seam-carving* sobre ella.

Metodología de Trabajo

En primer lugar, para poner a prueba esta hipótesis es necesario primero definir a qué llamamos objeto en este contexto. Así, a los fines de este experimento, un objeto es una forma geométrica claramente distinguible de un fondo de píxeles homogéneos en color.

Además, al igual que en el experimento anterior, para garantizar la integridad de los datos y minimizar el impacto de variables externas, se definió el siguiente protocolo:

- Muestreo**:** Se realizaron 10 ejecuciones independientes de una 100 iteraciones para PD y 1 para BT por cada imagen de cada dataset, calculando el promedio de los tiempos obtenidos. Se utilizó la biblioteca *<chrono>*, tomando marcas de tiempo antes y después del bloque de iteraciones mediante *std::chrono::high_resolution_clock*.
- Entorno**:** Se utilizó una Notebook Lenovo Thinkpad T14 i7-1255U, 16 gb RAM, SSD 512GB. La computadora estuvo permanentemente en carga de batería. Además de Visual Studio Code había una ventana abierta de Chrome con una hoja de cálculo para guardar los resultados. El sistema operativo es Windows 11. El software es C++ con GCC 15.2.0.
- Algoritmos**:** Se utilizó Programación Dinámica (PD) con el primer dataset, por su cantidad de píxeles, y Backtracking con el segundo dataset.
- Instancias**:**
  I. **PD:** Una progresión de imágenes (800x586 píxeles) con desde 0 a 5 objetos y 10 de estos, siendo los objetos cuadrados amarillos sobre un fondo sólido violeta. Los objetos no se añaden siguiendo un orden en particular.
  II. **BT:** Una progresión de imagenes (16x16 píxeles) con desde 0 a 7 objetos siendo los objetos cuadrados naranjas sobre un fondo sólido negro. Los objetos se añaden a la derecha del primer objeto, exceptuando el 7mo objeto.

Experimentos

Registramos el tiempo de ejecución del algoritmo de *seam-carving* para cada imagen de ambos datasets utilizando Backtracking o Programación Dinámica según habíamos predefinido en la metodología de trabajo. Nótese que discriminamos de esta medición al corte de la costura de mínima energía.

Resultados y discusión

Resultados de los experimentos:

#objetos	0	1	2	3	4	5	10
Tiempo promedio	26,95 s	26,94 s	24,38 s	26,05 s	26,92 s	26,17 s	27,42
SD(Tiempo)	2,97 s	2,35 s	0,81 s	2,30 s	3,55 s	1,95 s	5,87

Tabla I: Tiempo promedio y desvío estándar para 10 ejecuciones de cada imagen del dataset para programación dinámica.

#objetos	0	1	2	3	4	5	6	7
Tiempo promedio	0,16 ms	0,38 ms	0,62 ms	1,22 ms	7,82 ms	8,07 ms	13,07 ms	156,97 ms
SD(Tiempo)	0,04 ms	0,11 ms	0,14 ms	0,15 ms	0,38 ms	0,47 ms	1,49 ms	15,87 ms

Tabla II: Tiempo promedio y desvío estándar para 10 ejecuciones de cada imagen del dataset para Backtracking.

Análisis de los resultados:

Por un lado, al igual que en el anterior experimento, no encontramos diferencias significativas en el tiempo de ejecución de imágenes con distinta cantidad de objetos para programación dinámica (Tabla I). Este comportamiento era esperable dado que el algoritmo sigue una lógica de *tableau-filling* (llenado de tabla), el recorrido bottom-up se realiza de la misma forma independientemente del valor de las energías de la matriz. Su complejidad temporal está ligada exclusivamente a las dimensiones de la entrada (O(HxW)), por lo que no importa el contenido de la imagen.

Por otro lado, en Backtracking, aunque habíamos pensado que al algoritmo le sería difícil cortar imagenes de únicamente un fondo de color sólido porque todos los píxeles son identicos, analizando el bajo tiempo de ejecución promedio obtenido (Tabla II) de 0,16 segundos, notamos que este escenario es ideal para nuestra implementación. Al analizarlo, vemos que esto se debe a que la matriz de energías en estos casos es simplemente una matriz nula de n x m, y así el primer seam que se termina de recorrer tiene energía = 0. Por lo tanto, nuestra poda por optimalidad actúa con todos los otros seams, que tendrán al comienzo energía = 0 y así tendrán una energía mayor o igual a la mejor ya encontrada.

Además, un resultado inesperado es la diferencia en tiempos de ejecución entre la imagen con 6 objetos y la imagen con 7 (Tabla II). El tiempo promedio es 12 veces mayor que en la imagen con 6 para esta última. Ningún otro par de imágenes consecutivas presenta este tamaño de factor de diferencia. Frente a esto, analizamos más detenidamente las imágenes y notamos que el 7mo objeto lo habíamos añadido ligeramente a la izquierda del primer objeto de forma tal que algunos de los píxeles de la primera columna eran ahora píxeles borde[2]. Como en nuestra implementación Backtracking se ejecuta de izquierda a derecha, reflexionamos que tal vez espejando la función, es decir, no obstruyendo esos primeros caminos con píxeles de energía alta, el tiempo de ejecución bajaría nuevamente. Al comprobar que efectivamente esto sucedía propusimos una nueva hipótesis.

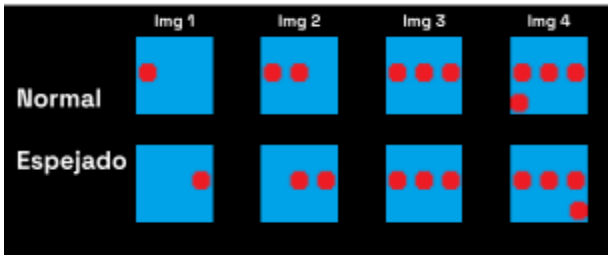
Hipótesis actualizada

A partir de la interpretación de los anteriores experimentos, propusimos una nueva hipótesis. Queremos ver si cuando existen objetos principales a izquierda en la imagen utilizada, el tiempo de ejecución de nuestra implementación de *seam-carving* en Backtracking será mayor que en esa misma imagen espejada.

Metodología de Trabajo

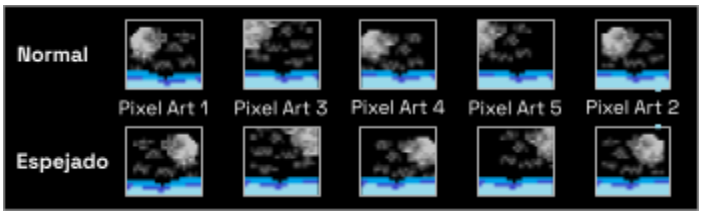
En términos generales mantuvimos la metodología anterior. Los únicos cambios son las siguientes:

- Muestreo**:** Se realizaron 11 ejecuciones independientes de una iteración por cada imagen de cada dataset, calculando el promedio y desvío estándar de los tiempos obtenidos.
- Algoritmos**:** Se utilizó únicamente Backtracking por los motivos ya discutidos anteriormente.
- Instancias**:**



I. Una progresión de 4 imágenes (26x26 píxeles) con entre 1 y 4 objetos (círculos rojos sobre un fondo sólido azul y sus espejos). Los objetos se añaden a la derecha del anterior objeto, exceptuando el último objeto en la versión normal.

[2]Pixeles con adyacencia a al menos otro píxel de valores RGB diferentes de los propios; son los de mayor energía.



II. Un set de 5 imágenes (24x22 píxeles) de tipo "Pixel Art" de una luna en la noche con nubes y un mar de tres tonos, y sus espejos.

Experimentos

Registramos el tiempo de ejecución del algoritmo de *seam-carving* para cada imagen de ambos datasets utilizando Backtracking según habíamos predefinido en la metodología de trabajo. Nótese que nuevamente discriminamos de esta medición al corte de la costura de mínima energía. El Dataset I fue confeccionado para, al igual que en los experimentos anteriores, proponer un escenario con las características mínimas para testear nuestra hipótesis. Utilizamos un fondo sólido y círculos como objetos añadiendose desde el extremo izquierdo (en la versión normal). El Dataset II fue creado para intentar proponer un contexto experimental más realista que los anteriores. En estos, hay un objeto "luna" en el extremo izquierdo de las versiones normales. Como Backtracking solo funciona en tiempos razonables con imágenes de muy baja resolución, decidimos que los *Pixel Arts* eran lo más cercano a una fotografía o una imágen con más detalle factibles de ser utilizados como dataset. En general, no recomendamos usar Backtracking en contextos con mayor resolución que los de un *Pixel Art* de baja resolución, ya que los tiempos de ejecución crecen rápidamente en función de la cantidad de filas y columnas.

Resultados y discusión

Resultados de los experimentos:

	Img 1	**Img 2**	**Img 3**	**Img 4**
$Media_{normal}$ ±SD_{normal}	22,37±6,17	21,53±6,38	24.46±5,49	21.94±6,34
$Media_{esp}$ ±SD_{esp}	0,51±0,14	0,47±0,15	20,66±5,38	21,32±5,82
$\Delta Media_{normal}$-$Media_{esp}$(%)	97,72	97,79	15,53	2,83

Tabla III: Tiempo promedio y desvío estándar, y diferencia porcentual entre las medias de una imagen normal y su espejo, para 10 ejecuciones de cada imagen del dataset I.

	Pixel Art 1	**Pixel Art 2**	**Pixel Art 3**	**Pixel Art 4**	**Pixel Art 5**
$Media_{normal}$ ±SD_{normal} (s)	65.03±5.63	13.20±2.49	132.63±11.05	246.42±59.11	19.09±1.39
$Media_{esp}$ ±SD_{esp}(s)	15.78±3.19	13.16±2.49	87.83±17.12	189.64±60.43	2.69±0.35
$\Delta Media_{normal}$-$Media_{esp}$(%)	75,73	0,30	33,78	23,04	85,92

Tabla IV: Tiempo de ejecución promedio, desvío estándar, y diferencia porcentual entre las medias de un Pixel Art normal y su espejo, en segundos para 10 ejecuciones de cada imagen del dataset II.

Análisis de los resultados:

Los resultados de la experimentación con el dataset I son coherentes con la hipótesis planteada. Esto se debe a que en las dos primeras imágenes, el lado izquierdo de la imagen normal no ofrece un camino rápido (sin píxeles borde) para el Backtracking. En ambos casos, las imágenes presentaban objetos en el extremo izquierdo en la versión normal. A causa de esto, en esos casos, la versión espejada se ejecuta en un tiempo notablemente menor: $Media_{esp}$ es un 92,72% menor que $Media_{normal}$ para "Img 1" y 97,79% para "Img 2" (en términos de $Media_{normal}$). Además, la tercera imagen es simétrica por lo que su espejo se comporta de la misma

manera. Esto se ve en la similitud de sus medias, aún más si consideramos sus desvíos estándar. Es esperable que en esa imagen no se noten diferencias en los tiempos de ejecución. También lo es que aumente el tiempo de ejecución entre la "Img 2" y la "Img 3" espejada. Esto se debe a que, en la última, se añade un objeto en el extremo izquierdo de la imagen (versión espejo). Como las costuras se recorren de izquierda a derecha, la presencia de una nueva zona de altas energías allí (como la que se forma en el contraste entre este objeto y el fondo) impide que se poden rápidamente las costuras subsiguientes (en regla con la poda que descarta las costuras cuya energía actual sea mayor o igual a la mejor energía ya conseguida). Por último, en la cuarta imagen ambas presentaciones (normal y espejada) tienen tiempos de ejecución muy similares. Además no hay una gran diferencia en los tiempos de ejecución de la "Img 3" a la "Img 4" para ninguna de las versiones. Creemos que esto se debe a que el cuarto círculo no añade mayores complicaciones a la ejecución del algoritmo de las que ya supone el círculo en el extremo izquierdo.

Por otro lado, los resultados de la experimentación con el dataset II también apoyan fuertemente la hipótesis planteada. Como se observa en la Tabla IV, exceptuando el "Pixel Art 2" todas las imágenes se ejecutaron en un tiempo considerablemente menor para el caso espejado: Media_esp es un 75,73%, 33,78%, 23,04%, y 85,92% menor que Media_normal para "Pixel Art 1", "Pixel Art 3", "Pixel Art 4", y "Pixel Art 5", respectivamente (en términos de $\text{Media}_{\text{normal}}$). En todos estos casos, la versión normal presentaba un gran objeto en el extremo izquierdo ("la luna"), mientras que allí en la versión espejada hay un fondo sólido ("el cielo nocturno"). Nótese que el caso del "Pixel Art 2" es particular porque allí en ambas versiones (espejada y normal) el primer *seam* que será recorrido, el del extremo izquierdo, es idéntico y es uno de los de mínima energía (el otro es el último *seam* que será recorrido). Aunque no sea simétrica, como "Img 3" del dataset I, los extremos de la imagen, que son de mínima energía, sí lo son. Es por esto que estas imágenes no presentan diferencias significativas en su tiempo de ejecución. En ambas presentaciones se encuentra una solución óptima rápidamente y se podan las costuras subsiguientes.

Conclusiones

En programación dinámica el contenido de la imagen en tanto a objetos es irrelevante. Como ya mencionamos anteriormente, su complejidad temporal está ligada exclusivamente a las dimensiones de la entrada ($O(A \cdot B)$ con A = cantidad de filas y B = cantidad de columnas). La hipótesis inicial sobre la relación entre el tiempo de ejecución y la cantidad de objetos presentes en la imagen tampoco se ve respaldada por los resultados obtenidos en Backtracking. En la mayoría de los casos, los tiempos de ejecución se mantuvieron similares a pesar de variar la cantidad de objetos, tanto en Programación Dinámica como en Backtracking. La única diferencia significativa observada (entre "Img 6" e "Img 7") no parece explicarse por la cantidad de objetos en sí, sino por su ubicación dentro de la imagen.

En cambio, se encontró evidencia consistente a favor de una hipótesis actualizada: el posicionamiento de objetos en el extremo izquierdo influye en el tiempo de ejecución del algoritmo de *seam-carving* en Backtracking. Dado que en nuestra implementación las costuras se recorren de izquierda a derecha, la presencia de zonas de alta energía en el extremo izquierdo (por ejemplo, bordes generados por el contraste entre un objeto y el fondo) dificulta la poda temprana y aumenta el tiempo de ejecución. Por el contrario, cuando el extremo izquierdo presenta zonas de baja energía (como un fondo homogéneo), el algoritmo encuentra rápidamente soluciones óptimas y reduce significativamente el espacio de búsqueda. En este sentido, para mejorar el rendimiento de Backtracking, resulta conveniente trabajar con imágenes que tengan fondos sin detalles a la izquierda y objetos hacia la derecha. Alternativamente, es posible espejar la imagen antes de ejecutar el algoritmo y revertirla luego, o bien modificar el orden en el que se recorren las costuras comenzando desde la derecha.

Conclusiones finales

Los sucesivos experimentos conducidos nos permiten esbozar las siguientes conclusiones:

1. Los resultados indican que, en programación dinámica, el tiempo de ejecución es independiente del contenido de la imagen. En efecto, su complejidad temporal depende únicamente de las dimensiones de la entrada ($O(A \cdot B)$, con A filas y B columnas).
2. Por el contrario, el rendimiento del algoritmo de Backtracking es sumamente dependiente del contenido de la imagen debido a la poda de optimalidad. Imágenes cuyo contenido ofrezca un *seam* óptimo que se pueda hallar más temprano serán procesadas mucho más rápidamente en Backtracking que aquellas que no.
3. En cuanto a la comparación entre lenguajes, los resultados muestran de manera consistente que C++ supera a Python en la ejecución de todos los algoritmos implementados. Esta diferencia es primordialmente notoria en Programación Dinámica, donde la gran cantidad de operaciones elementales magnifica las ventajas del lenguaje.
4. En relación con el experimento de imágenes a color versus blanco y negro, los resultados también contradicen nuestra hipótesis inicial. En Backtracking, las imágenes en blanco y negro resultaron consistentemente más rápidas. Una posible conjetura es que una menor variabilidad en los valores de energía (producto de la similitud entre los canales RGB) favorece la aparición temprana de soluciones de baja energía, lo cual activa la poda de manera más agresiva y reduce el espacio de búsqueda. De todas formas, esta interpretación debe tomarse con cautela debido a las limitaciones del experimento.
5. En esa misma línea, observamos que el método de conversión a blanco y negro podría estar influyendo en los resultados. El testing realizado sugiere que el sitio utilizado aplica un criterio de luminancia, lo cual no sólo reduce la información cromática sino que también introduce una redistribución de intensidades. Esto podría "suavizar" o "aplanar" la matriz de energía y favorecer el comportamiento mencionado anteriormente. Sin embargo, también es posible que, en otros casos, este mismo proceso complejice la matriz (y, por lo tanto, retrase la ejecución). Dado que no contamos con información precisa sobre la implementación (y considerando además que incluso para una imagen originalmente en blanco y negro la conversión también resultó más rápida) esta interpretación es sólo tentativa.
6. Consideramos que una posible mejora del trabajo sería tener un mayor control sobre la generación de las imágenes utilizadas en el experimento de la conversión a blanco y negro. Implementar un método propio permitiría aislar mejor las variables en juego y obtener conclusiones más robustas.
7. Nuestra hipótesis inicial sobre la relación entre el tiempo y la cantidad de objetos resultó ser errónea. No encontramos evidencia suficiente en nuestros experimentos para descartar que no haya relación entre estas dos variables. Los tiempos de ejecución dieron similares para diferentes cantidades de objetos para todas las imágenes y algoritmos excepto entre la "Img 6" y la "Img 7". Sin embargo, esta diferencia no se explica por el objeto agregado sino por el posicionamiento del mismo.
8. Encontramos evidencia a favor de nuestra hipótesis actualizada sobre el posicionamiento de objetos en el extremo izquierdo y su relación con el tiempo de ejecución de *seam-carving* en nuestra implementación de Backtracking. Como las costuras se recorren de izquierda a derecha, la presencia de una zona de altas energías (como la que se forma en el contraste entre un objeto y un fondo) en el extremo izquierdo de la imagen impide que se poden rápidamente las costuras subsiguientes. Por el contrario, si allí hay una zona de energías bajas (como la que se forma en presencia de un fondo sólido), el algoritmo podará sin mayores inconvenientes. Por lo tanto, en Backtracking, es recomendable utilizar el algoritmo en imágenes con zonas de fondo a izquierda y objetos a derecha. También en una imagen con las características opuestas es posible espejarla, ejecutar *seam-carving,* y espejarla nuevamente. Otra opción es invertir el orden en el que se recorren las costuras, comenzando por la derecha.